

CoachOS

Practice-management SaaS for independent coaches and small coaching firms. One workspace-scoped system of record for clients, sessions, the sales pipeline, invoicing, documents and email, plus a branded self-serve portal for the coach's own clients.

Reviewed from source on October 8, 2026 · production: coachos.rass-consulting.com · supersedes the architecture sections of `system_design.md` (Aug 29) and the "no AI" note in `CLAUDE.md`

12	42	4	8	7
Django apps	data models	identity scopes	scheduled jobs	external services

Contents

1. Product and roles	11. Client portal
2. System architecture	12. Library, documents and contracts
3. Request path and tenant isolation	13. AI session-note drafts
4. Identity and access	14. Integrations
5. Feature map	15. Background jobs
6. Data model	16. Public demo
7. Scheduling and calendar	17. Platform admin and observability
8. Billing and payments	18. Operations
9. Sales pipeline	19. Known gaps and risks
10. Email and notifications	

1 Product and roles

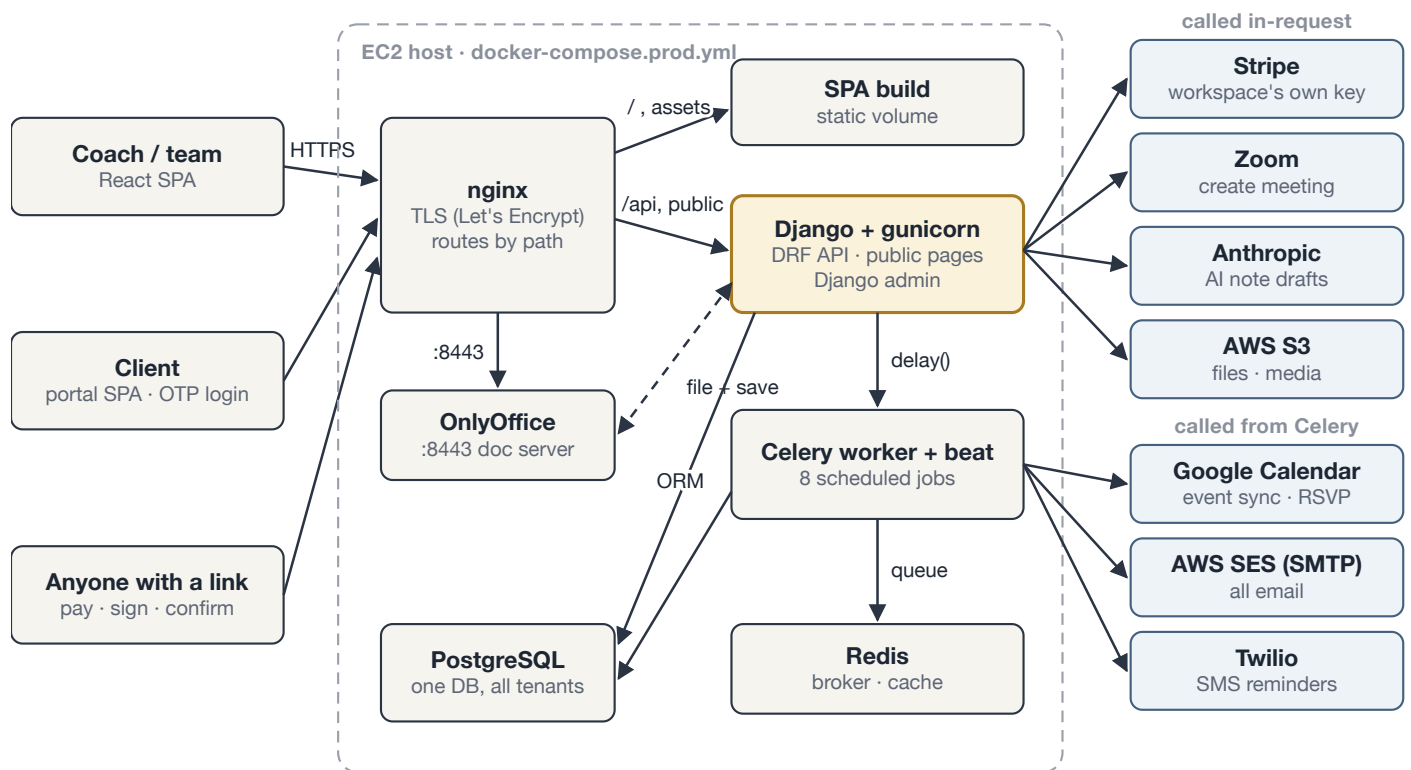
Coaches today juggle notes in Notion or Docs, Calendly plus Google Calendar, Stripe plus QuickBooks, and a pipeline spreadsheet. CoachOS replaces that set with one system per **workspace**, which is the tenant boundary. A user belongs to exactly one workspace. Their role sets a baseline, and the owner can widen or narrow it per section.

Role	Scope	How it's enforced
business_owner	Everything in the workspace: billing, team, integrations, Email Communication	Role checks on viewsets; owner-only routes via <code>RoleRoute</code>
coach	Own clients and schedule; more sections if granted	<code>TabPermission</code> view / edit / delete per section
assistant · limited	Operational, then further-restricted access	Same per-tab overrides
platform_admin	Crosses workspaces: CoachOS's own console	<code>IsPlatformAdmin</code> on <code>/api/superadmin/*</code>
portal_client	The coach's client, in the self-serve portal only	Separate JWT scope (see §4)

The frontend mirrors these rules: `TabRoute` and `RoleRoute` wrap every gated page, so a role that would get a 403 never sees the UI in the first place.

2 System architecture

Everything runs on a single host. Tenancy is purely an application and data-layer concern; there is no per-tenant infrastructure.

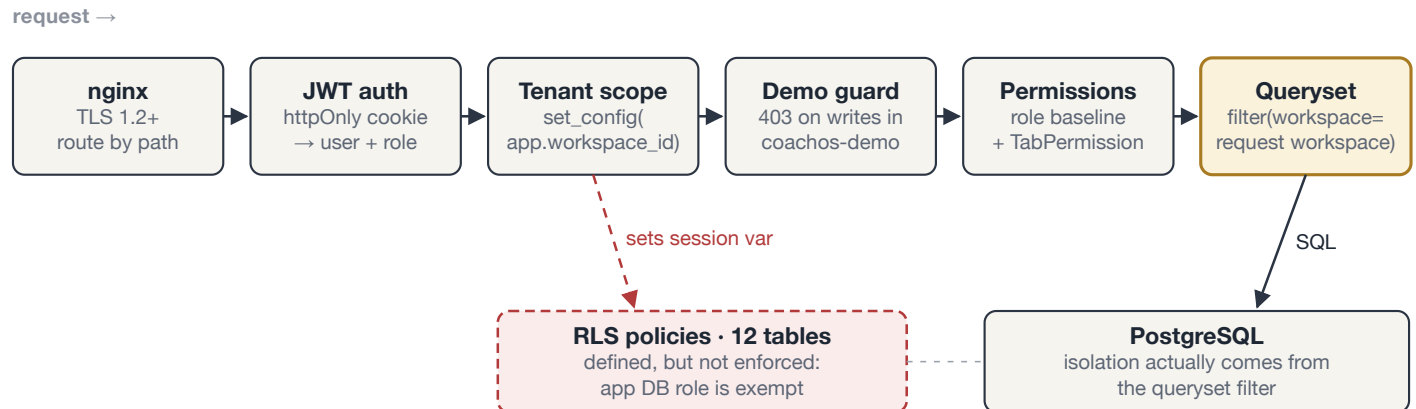


Everything runs on one EC2 host. `nginx` is the only public entry point: it serves the built SPA, proxies `/api` and the public token pages (`/invoices/pay`, `/contract/sign`, `/session/...`, `/accounts`, `/django-admin`, `/static`) to gunicorn, and exposes OnlyOffice on `:8443`. Payments, AI drafts, Zoom and file storage happen inside the request; calendar sync, email and SMS go through Celery. Google also calls back in, via push notifications to `/api/webhooks/google-calendar/`.

Container	Role	Notes
nginx	TLS termination, path routing, static files	Config bind-mounted from <code>nginx/nginx.conf</code> ; <code>^~ /static/</code> must win over the SPA's <code>*.css *.js</code> rule
backend	Django 5 + DRF on gunicorn (2 workers)	Entrypoint runs migrations and <code>collectstatic</code> before serving
celery · celery-beat	Async tasks; <code>DatabaseScheduler</code>	Schedule defined in <code>CELERY_BEAT_SCHEDULE</code> , live copy in Django admin → Periodic tasks
db · redis	PostgreSQL (local only); Celery broker and Django cache	Production data lives on AWS RDS ; the compose <code>db</code> container there is unused. Task results stored in Postgres (<code>django-db</code>)
onlyoffice	Document Server for in-browser Word/Excel/PPT	Separate server block on <code>:8443</code>
frontend	Builds the React 18 + Vite SPA into a shared volume	Zustand auth store, React Query, Tailwind, FullCalendar, Recharts, driver.js

3 Request path and tenant isolation

All workspaces share one database. A request becomes tenant-scoped by passing the chain below. Each workspace-owned model inherits `WorkspaceModel` and carries a `workspace` foreign key.



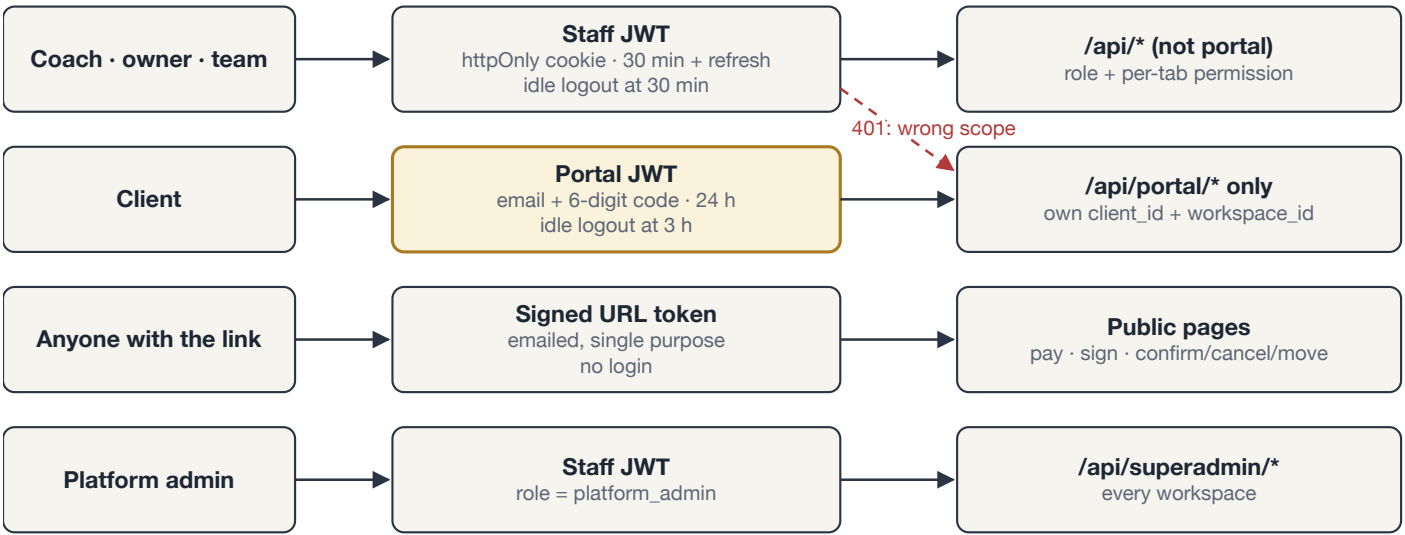
One API request, left to right. Every step can reject the request. Tenant isolation is enforced by the last step: each viewset filters its queryset by the caller's workspace. The middleware also sets a Postgres session variable for row-level-security policies, but Postgres skips them: locally the app connects as a superuser, and on RDS as `coachos_admin`, which owns the tables while none of them use `FORCE ROW LEVEL SECURITY` (verified 2026-10-08). The portal scope also sets `app.client_id`.

Design risk: only one isolation layer. The RLS policies (`tenant_isolation`, plus `portal_isolation` on goals) look like a second line of defence, but they don't run: in production the application connects as `coachos_admin`, which owns the tables, and the tables don't use `FORCE ROW LEVEL SECURITY`, so Postgres exempts the owner. Only 12 of 34 workspace tables have policies at all. A single viewset that forgets its workspace filter would leak across tenants. Fix (planned in `PHASE2.md`, P2-2/P2-3): a non-owner runtime role, `FORCE + WITH CHECK` on every workspace table, and an explicit tenant context for Celery tasks and webhooks, which run without the request middleware.

Writes are scoped too (since 2026-10-08). Reads were always filtered by workspace, but writable foreign keys (`client`, `coach`, `deal`, ...) used DRF's default lookup over every workspace's rows, so workspace B could create an invoice, deal or session against workspace A's client. Every serializer with a writable FK now mixes in `WorkspaceScopedSerializerMixin` (`apps/accounts/tenancy.py`): the FK resolves only within the caller's workspace and, for non-owners, only to rows they can already read (a coach can pick only their own clients). It fails closed with no request in context. Login also refuses a non-admin user with no workspace instead of attaching them to the oldest one. `backend/test_tenant_isolation.py` covers cross-workspace and coach-vs-coach reads and writes, portal client-vs-client, and includes a guard test that fails if a future serializer adds an unscoped writable FK.

The `DemoWorkspaceReadOnlyMiddleware` sits right after the tenant middleware. It is real Django middleware, not a DRF default permission, because most viewsets declare their own `permission_classes`, which would silently replace a global default. Login, logout, refresh, demo-lead capture and the portal demo-login are exempt.

4 Identity and access



Four identities, each issued differently and reaching a different slice of the API. A staff token carries no `client_id` and a portal token carries `role=portal_client`, so neither is accepted by the other side's authentication class. Public links carry a signed, single-purpose token instead of a session.

Control	Setting
Throttles	anon 200/h · user 2000/h · login 10/min · password reset 5/min · register 5/h · portal code 5/min · demo lead 30/h · portal demo-login 20/min. Client IP comes from the last <code>X-Forwarded-For</code> hop nginx adds (<code>NUM_PROXIES = 1</code>), so a forged header can't dodge per-IP limits
Idle logout	Staff: warn at 15 min, log out at 30 min (all roles). Portal: warn at 2 h 45 m, log out at 3 h
Signup	Invite-only workspace creation through one-time <code>WorkspaceRegistrationToken</code> links issued by a platform admin; team members join through <code>WorkspaceInvitation</code> (failed invite emails retried every 5 min)
Audit	<code>AccessLog</code> records view/create/update/delete on clients, notes, goals, files and team, plus password changes and AI-draft requests
Secrets at rest	Stripe secret/webhook keys Fernet-encrypted with <code>FIELD_ENCRYPTION_KEY</code> (comma list, rotation-ready); OAuth tokens stored server-side only

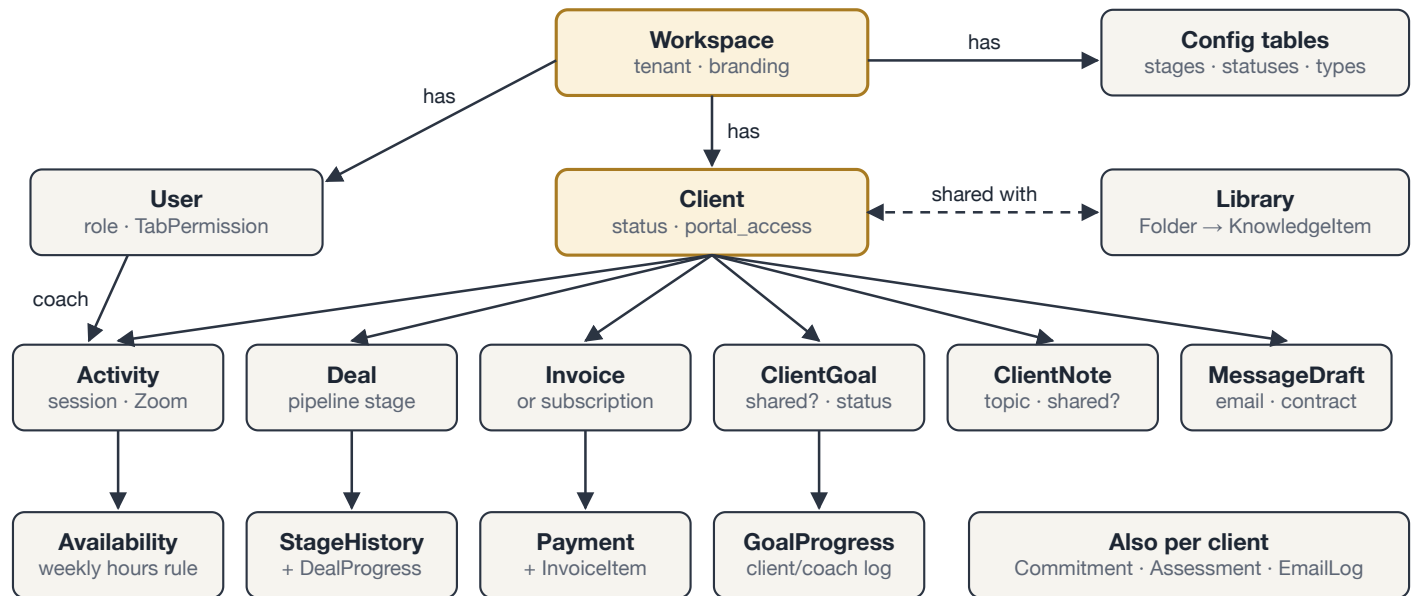
5 Feature map

Every user-facing capability, grouped by the Django app that owns it.

App	Features	Pages
accounts	Login/logout, password reset, invite-only registration, team invites and per-tab permissions, profile, Google Calendar and Zoom connect	Login, Register, AcceptInvite, Forgot/Reset, Team
clients	CRM with configurable statuses, tags, lead sources and affiliations; CSV import/export; session notes with topic, date, search and sharing (+ AI drafts); goals with progress and sharing; commitments; assessments in OnlyOffice; client messages and contract e-signing; coach availability rules; email log	Clients, NewClient, ClientDetail, EmailCommunication
activities	Calendar and list across 8 built-in, configurable activity types; recurrence (daily, weekly, biweekly, monthly, yearly; capped at 1 year ahead without an end date); overlap prevention; reminders; client confirm/cancel/reschedule links with slot picker; Confirm/Decline of proposals; Zoom links; Google Calendar 2-way sync	Calendar, Activities
pipeline	Kanban of deals across 8 configurable stages; stage history and progress; per-stage, per-recipient follow-up schedules	Pipeline, NewDeal
invoicing	One-time and subscription invoices; service catalog; PDF; Stripe Checkout pay links; partial payments; refunds; auto-Overdue; auto-archived Void; receipts	Invoices, NewInvoice, InvoiceDetail
reports	Revenue and outstanding reports, CSV export, dashboard KPIs	Reports, Dashboard
library	Folders and documents, OnlyOffice live editing, per-client sharing, PDF conversion	Library
portal	Client self-serve: overview, goals (own CRUD + mark coach goals complete), sessions + reschedule request, notes, files, invoices + pay	/client-portal
settings_app	Workspace branding and timezone; taxonomies; pipeline stages and follow-ups; services; email templates per use case; integrations; reschedule TTL	Settings (12 tabs)
feedback	Bug/feature tickets with comments between team and owner, plus a floating report button	Feedback list/detail
audit	Per-action audit trail (owner sees the last 100)	Settings
superadmin	Workspace list and drill-down, users, errors, audit, email diagnostics and resend, registration tokens, maintenance banners, manual platform invoices, demo leads	/admin

Public, unauthenticated pages: marketing home, privacy policy, terms, invoice pay + success, contract sign, and session confirm/cancel/reschedule.

6 Data model

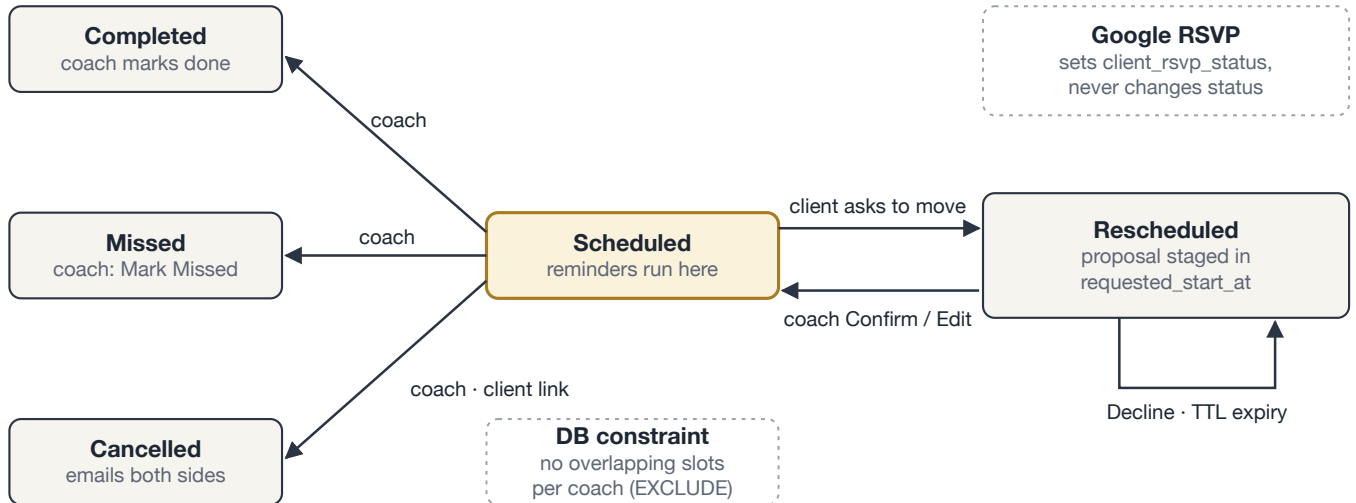


Client is the hub. Sessions, deals, invoices, goals, notes and outbound documents all hang off a client, and every one of these rows also carries `workspace_id`. Library items are shared with clients many-to-many. Workspace-level configuration (pipeline stages, client statuses, tags, lead sources, activity types, affiliations) lives in small per-workspace tables, seeded with built-in defaults on first use.

Supporting tables: `GoogleCalendarWatch` (per-coach push channel), `PortalLoginCode`, `EmailLog`, `AccessLog` and `AuditLog`, `ErrorLog`, `FeedbackTicket`, and platform-level `DemoLead`, `MaintenanceBanner` and `PlatformInvoice/Payment`.

7 Scheduling and calendar

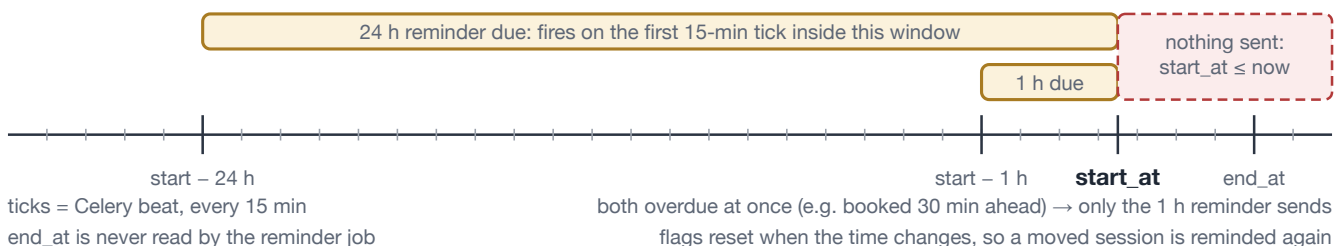
A session is an `Activity` with a coach, a client and one of 8 built-in, renameable types (appointment, session, call, task, training, travel, client communication, custom). Recurring series are written out as real rows linked by `recurrence_id`, up to `repeat_until` or one year ahead.



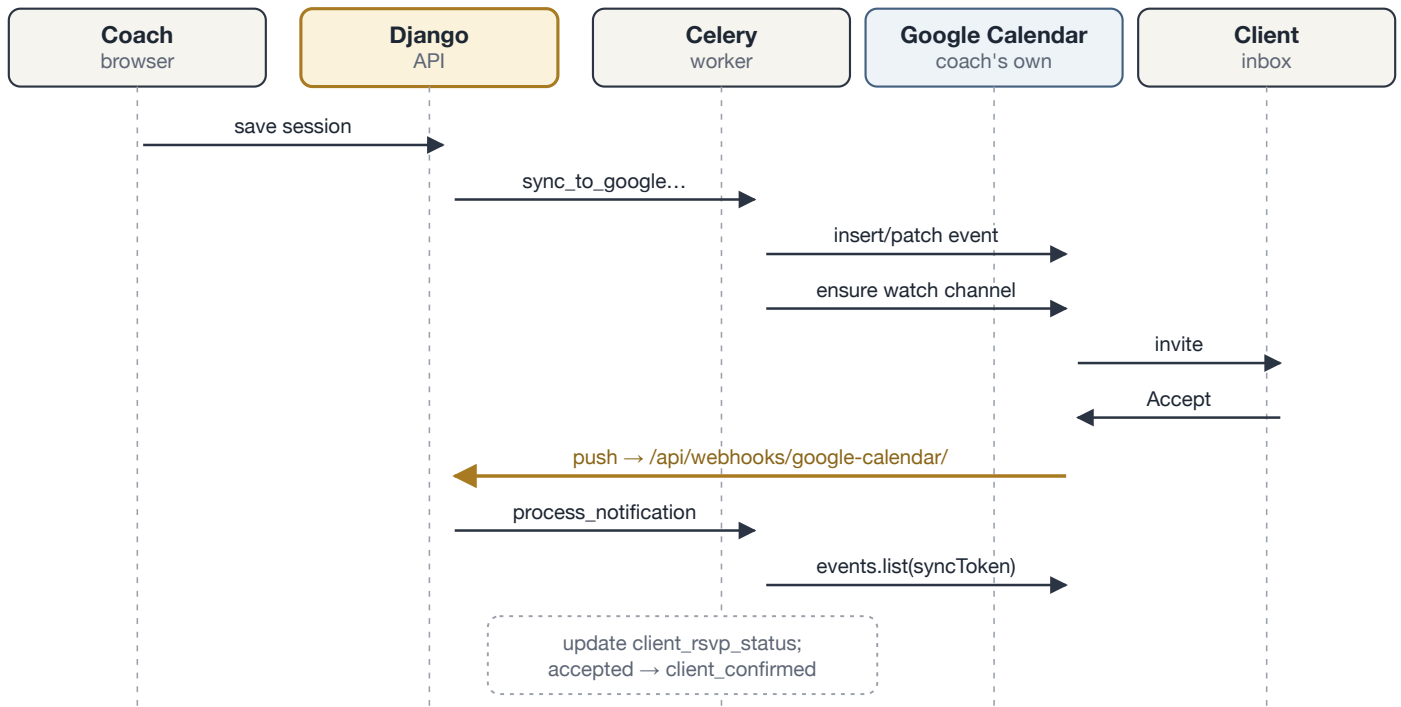
Session status lifecycle (`Activity.status`). A client-proposed time sits in `requested_start_at` and only takes effect when the coach confirms. Confirm re-checks that client's availability rules first, then moves the times, returns the session to `Scheduled`, resets the reminder flags and re-syncs Google Calendar. Decline, or the daily 04:00 expiry once the workspace's TTL passes (default 5 days), clears the proposal. A Postgres exclusion constraint rejects any write that would overlap another active session for the same coach. The `late` status exists in the model, but nothing sets it today.

Self-serve rescheduling

The emailed reschedule link offers a slot picker built from that client's `CoachAvailabilityRule`s. It checks busy time against the coach's confirmed sessions *and* every other client's pending proposal, so two clients can't claim the same slot. Without availability rules, the link falls back to a free-text request. The portal's reschedule request is always free text.



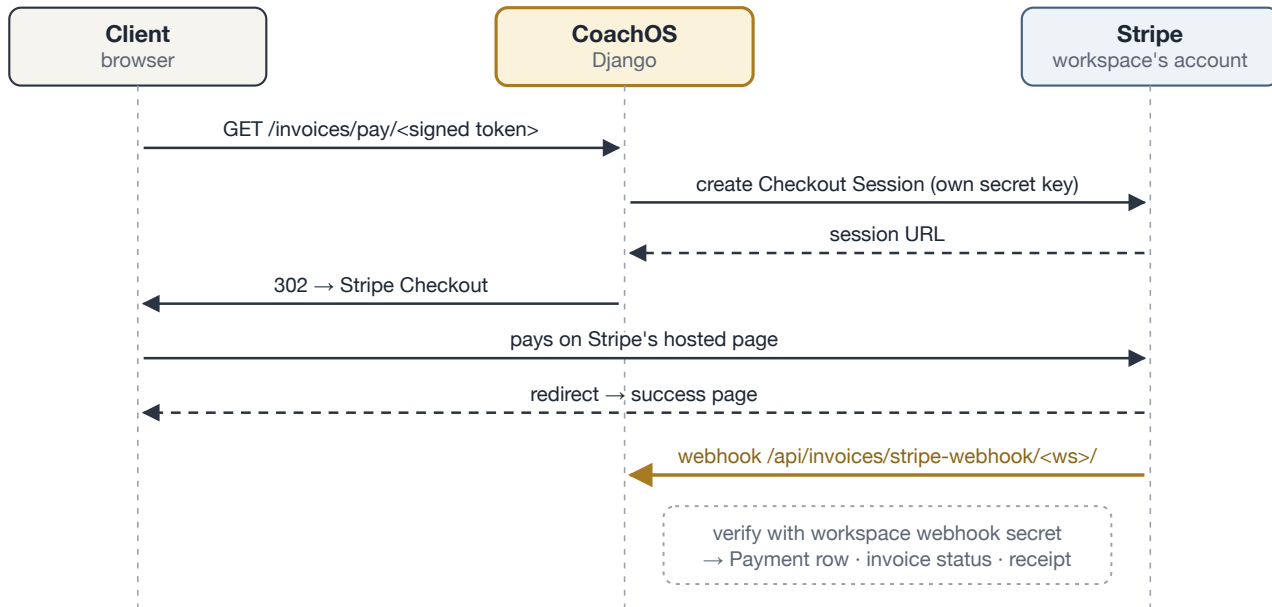
Reminder timing for one session (`tasks/reminders.py`). The rule is "due by", not "due at": if Celery misses ticks, the reminder still sends late, as long as the session hasn't started. Only `status = scheduled` sessions qualify. Email goes to the client (plus a coach copy); SMS goes out if the client has a phone number. `reminder_24h_sent` / `reminder_1h_sent` prevent duplicates, with a Redis key as a second guard.



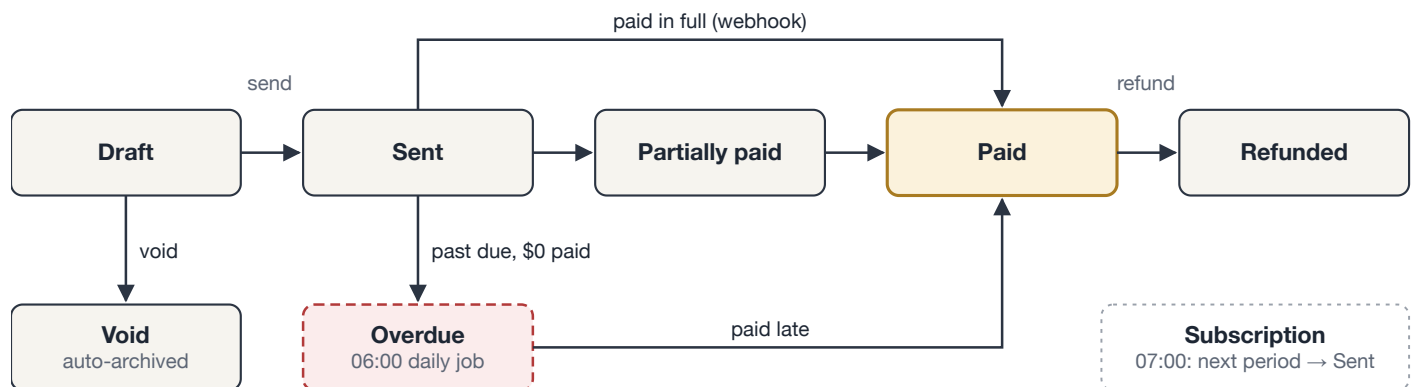
Two-way Google Calendar sync. The coach's own calendar holds the event, with the client as an attendee. When the client RSVPs, Google pushes a change notification. CoachOS then pulls only the delta with a stored sync token and records the RSVP. Watch channels expire, so a 03:00 job renews them. Each coach connects separately through OAuth. While the Google app is unverified, refresh tokens die after 7 days (see Known gaps).

Zoom. From Calendar or a client record, the coach clicks to create a meeting. The call happens in the request and stores `join_url` on the session. The integration subscribes to no Zoom webhooks and keeps no other Zoom data.

8 Billing and payments



Getting paid. Card data only ever touches Stripe, and money lands in the workspace's own Stripe account. CoachOS holds the secret and webhook keys Fernet-encrypted in `Workspace.integrations`. The webhook URL includes the workspace id, so each signature is checked against that workspace's secret. Refunds go both ways: an in-app refund calls Stripe's Refund API, and a refund issued from the Stripe dashboard comes back through `charge.refunded`. Both paths dedupe on `stripe_refund_ids`.



Invoice lifecycle. Sending emails the PDF (WeasyPrint) with a Stripe pay link. The 06:00 job flips Sent invoices with no payment at all past their due date to Overdue. The 07:00 job generates and sends the next period's invoice for subscriptions with auto-send on. Line items can come from the workspace's service catalog.

CoachOS's own billing of its workspaces is a separate system. `dj-stripe` is installed, but `PlatformInvoice` billing is done by hand from the superadmin console.

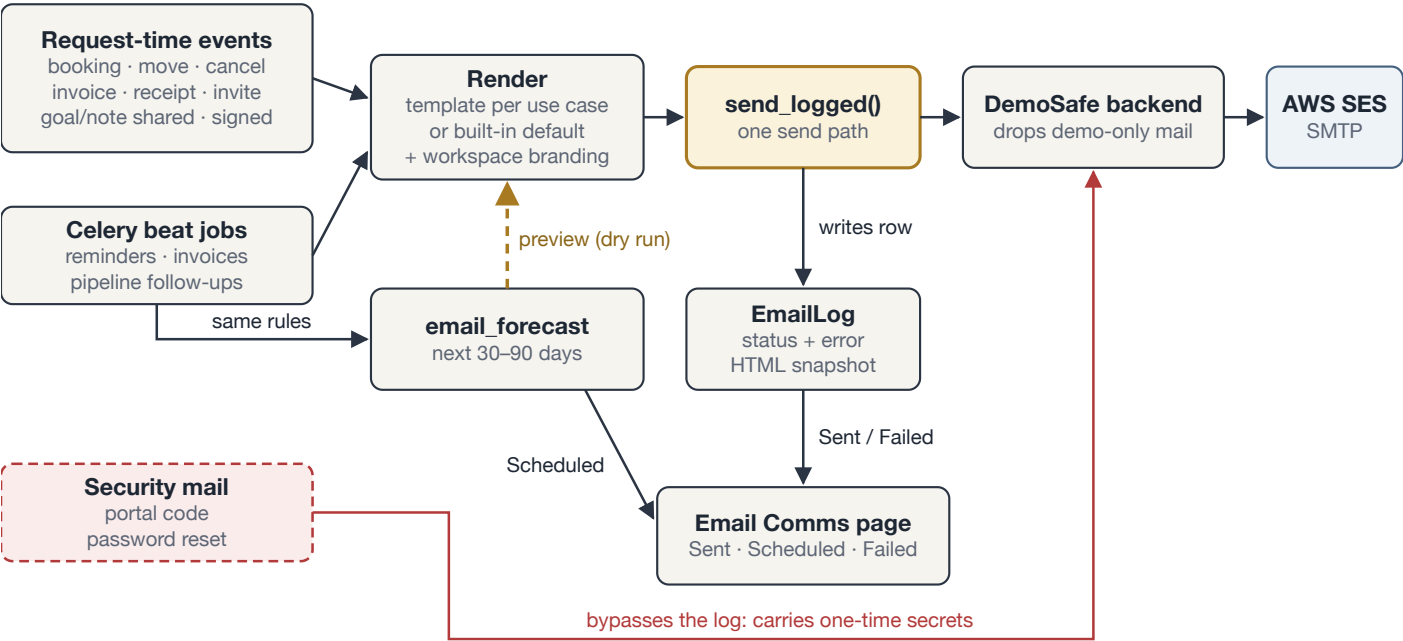
9 Sales pipeline

Deals move through workspace-configurable stages: Lead – New → Discovery Scheduled → Discovery Completed → Proposal Sent → Verbal Yes → Active Client, plus On Hold and Closed – Lost. Every move writes `StageHistory`. Creating a deal from the New Client form is opt-in.



Stalled-deal follow-ups. Each pipeline stage sets `follow_up_days`, an optional stop, and a separate on/off and schedule (daily, a weekday, or a day of the month) for each recipient. The beat job runs hourly and handles each workspace at 08:00 in its own timezone. The owner and coach receive the internal alert. The client receives a separate, editable check-in (`pipeline_client`). A per-deal stop date overrides the stage's stop.

10 Email and notifications

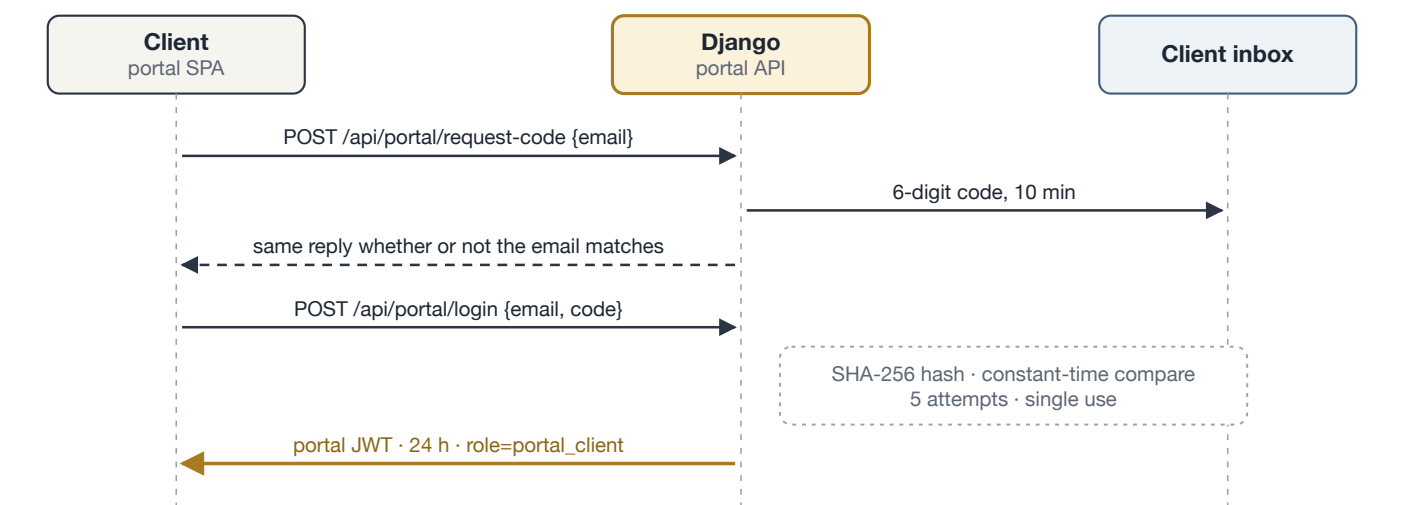


Every workspace email goes through `send_logged()`. It sends through the configured backend and records the result, so failed sends show up with their error and a new email type can't skip the log. The Scheduled tab is computed live from the same rules the beat jobs use, so it can't drift. A preview runs the real send code inside `capture_emails()` and a rolled-back transaction, which sends and saves nothing. Portal login codes and password resets skip the log on purpose. In production every message still passes the demo-safe backend.

Audience	Use cases (each editable in Settings → Emails, branded with logo)
Client	booking confirmation · reschedule · cancellation · 24 h and 1 h reminders · invoice · payment receipt · portal invite · client message/contract · goal shared · note shared · reschedule acknowledgement · decline · pipeline check-in
Coach / owner	session booked/updated/cancelled/reminder copies · client confirmed/cancelled/RSVP · reschedule request · payment failed · contract signed · pipeline follow-up
Team	team invite

SMS (Twilio, optional) sends session reminders when the client has a phone number. **Mail transport** is AWS SES over SMTP in production and Mailpit locally. In production, `DemoSafeEmailBackend` wraps SMTP.

11 Client portal



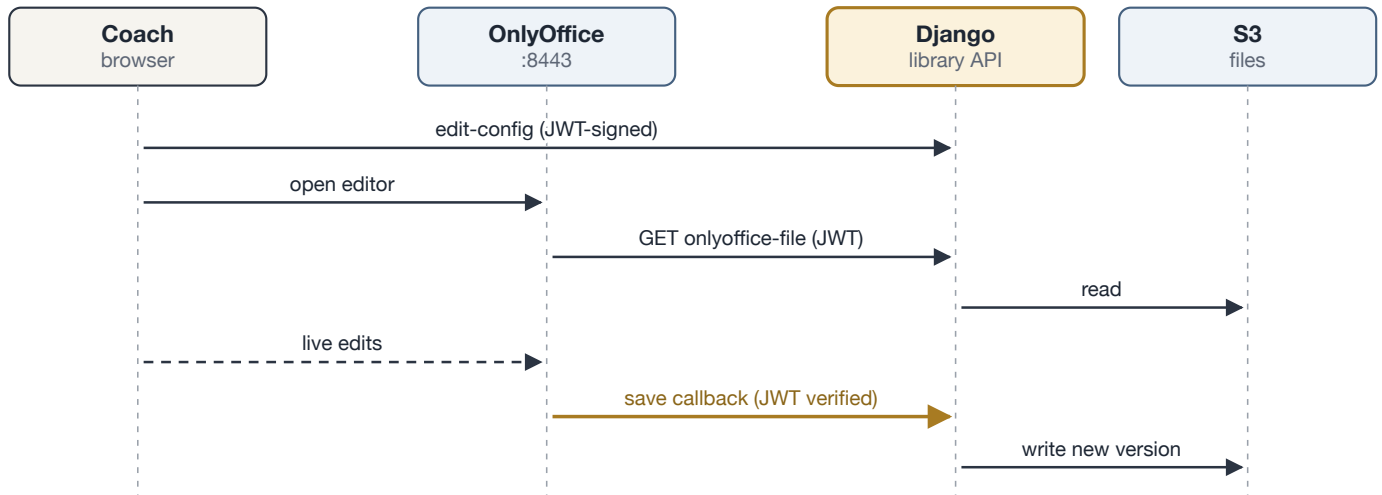
Client portal sign-in is two steps. Before 2026-09-30, the email alone was enough to sign in, which was an authentication bypass. Only clients with `portal_access` receive a code. Throttles are 5/min to request a code and 10/min to log in, and the identical reply prevents email enumeration. The code email is deliberately kept out of the coach-visible email log.

Tab	What the client can do
Overview	Goals due, upcoming sessions, outstanding invoices
Goals	Create, edit, delete and search their own goals; log progress; mark coach-shared goals complete or reopen them
Activities	Upcoming and past sessions (internal coach notes excluded); request a reschedule
Notes	Own notes (CRUD, search by topic/date); read notes the coach shared
Files	View and download library items shared with them
Invoices	View, download and pay through Stripe Checkout

Out of scope by design: assessments, messaging and contracts (contracts use their own signed link). Branding shows the workspace's logo and name.

Sign-in screens. Both logins share one layout: an editorial left column, a gold-topped card on the right, and an ombré background (warm paper to soft blue, `.auth-ombre`). A decorative canvas animation sits behind them, laid out from the real page elements (marked `data-flow-*`) so it never crosses text or the form, and the card title carries a gold North Star (`FlowStar`) that pulses as chains of dots arrive. The coach login (`NorthStarFlow`) fans lines in from the bottom-left corner to each feature bullet, then on to the card. The client portal (`GoldenThreadFlow`, modelled on LangSmith's sign-in page) fans curves in from the left edge into a straight gold line along the rule under the intro text, which rises into the card. Both hide on the stacked mobile layout and render one still frame under `prefers-reduced-motion`.

12 Library, documents and contracts



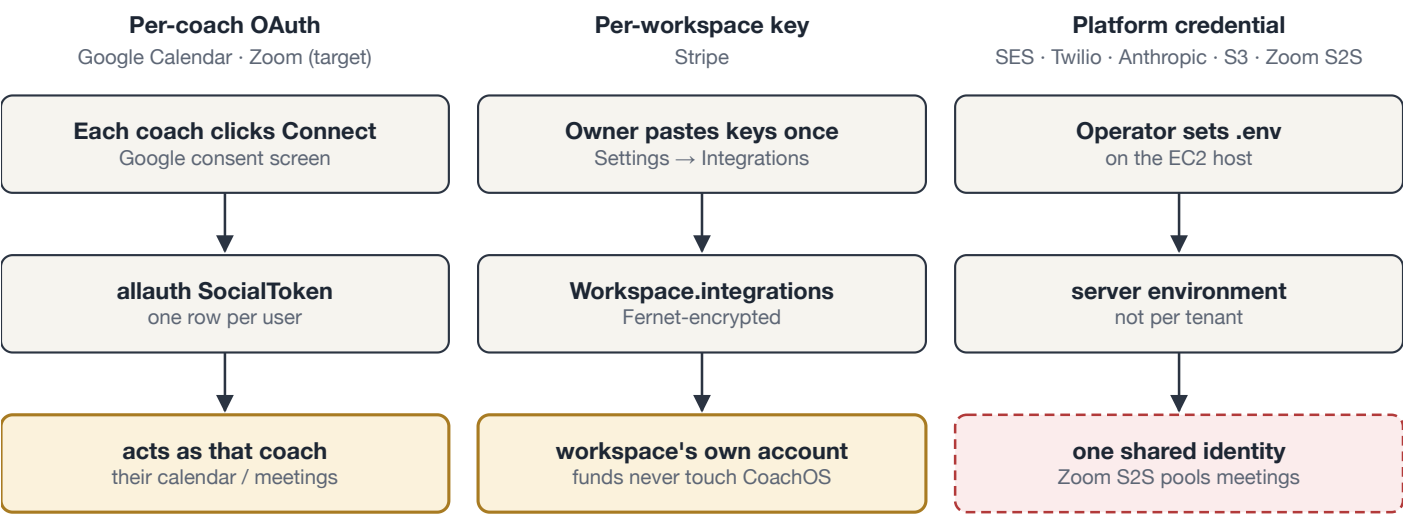
Library and assessment documents open in a self-hosted OnlyOffice server. Every hop between OnlyOffice and Django is signed with `ONLYOFFICE_JWT_SECRET` in both directions, because the document server calls Django directly without a user session. Items are shared with chosen clients and appear in their portal's Files tab.

Contracts and client messages. A `ClientMessageDraft` is emailed with optional attachments from S3. If it needs a signature, the client signs at a public `/contract/sign/<token>/` link. The draft then becomes `signed` and the coach receives a `contract_signed` notice.

13 AI session-note drafts

On a structured session note, **Suggest with AI** sends the coach's raw notes to Anthropic through `POST /api/clients/{id}/notes/suggest/` and returns a drafted Coach Reflection and Commitment. Nothing is stored server-side: the coach accepts or rejects each field and saves through the normal note endpoints. Each request is audit-logged. Without `ANTHROPIC_API_KEY`, the endpoint returns a clear 400; `AI_NOTES MOCK` is for development only.

14 Integrations



Three credential patterns, compared on whose identity the external call uses. Zoom is mid-migration: the per-coach OAuth path is built but blocked by a Zoom-side "Invalid redirect" error, so production runs the interim Server-to-Server path (`ZOOM_OAUTH_ENABLED=False`). That creates every workspace's meetings under one platform Zoom account. This is acceptable with one real workspace; it should be revisited before a second one joins.

Service	Pattern	Used for	Failure mode
Google Calendar	Per-coach OAuth	Event sync, RSVP push	Unverified app → weekly token expiry
Zoom	S2S today; OAuth built	Create meeting, store join URL	Shared account while OAuth is blocked
Stripe	Per-workspace key	Checkout, refunds, webhooks	Lost <code>FIELD_ENCRYPTION_KEY</code> = undecryptable keys
AWS SES	Platform SMTP	All email	Failures recorded in EmailLog
Twilio	Platform	SMS reminders	Optional; skipped without a phone number
Anthropic	Platform key	Session-note drafts	400 without a key
AWS S3	Platform (IAM role or keys)	Files, media, attachments	Presigned URLs expire after 30 min

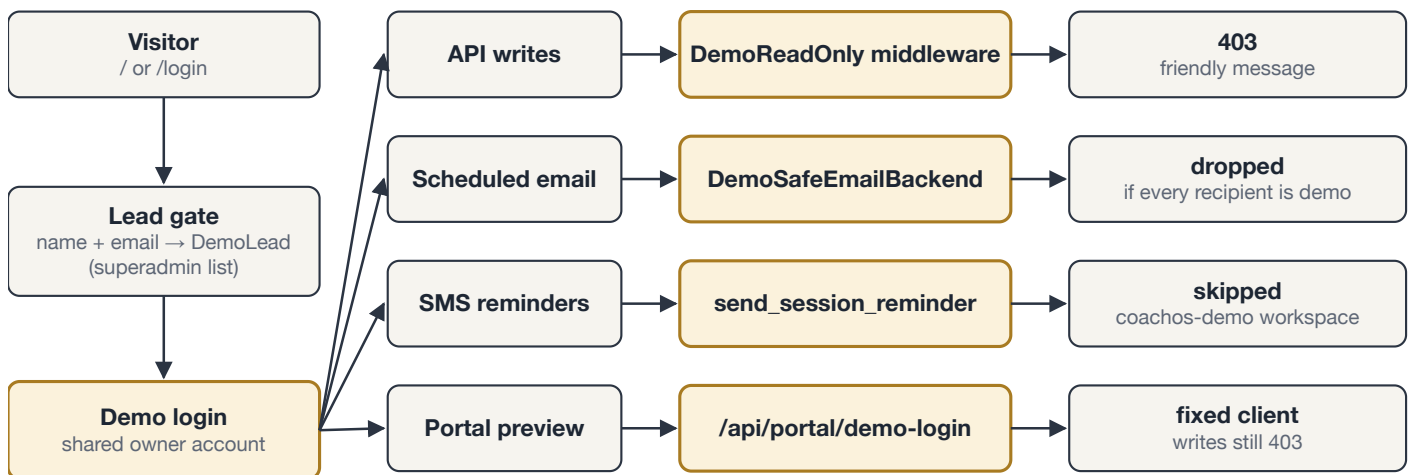
15 Background jobs

Job	When (UTC)	What it does
dispatch-activity-reminders	every 15 min	24 h / 1 h email + SMS reminders (§7)
retry-pending-invites	every 5 min	Re-sends team invites whose first email failed
check-site-health	every 30 min	Site reachability + TLS expiry; emails an alert (expiry alerts de-duplicated daily)
dispatch-pipeline-alerts	hourly	Acts at 08:00 in each workspace's timezone (§9)
renew-calendar-watch-channels	03:00	Renews expiring Google push channels
expire-stale-reschedule-requests	04:00	Clears proposals older than the workspace TTL
mark-overdue-invoices	06:00	Sent + unpaid + past due → Overdue
dispatch-subscription-invoices	07:00	Generates and sends the next subscription period

Where to see them: Django admin → **Periodic tasks** (live state, last run) and **Task results** (history). The code schedule is in `config/settings/base.py`. Logs: `make logs-celery` in production.

16 Public demo

"Log In as Demo User & Take the Tour" on `/` and `/login` runs a router-aware `driver.js` tour across every area: an intro, Dashboard, Clients, Pipeline, Calendar, Invoices, Email Communication ×3, Library, Reports, Settings ×2 and Team. It ends by opening the client portal as a seeded client. Real users get the same tour from the Dashboard, without the portal step.



The public demo shares one seeded workspace (`coachos-demo`) among all visitors. Four independent guards stop it from changing data or reaching real inboxes. Each covers a path the others can't see: HTTP middleware never sees Celery jobs, and the email backend never sees SMS. Re-running `seed_demo_workspace` resets the data and is idempotent.

17 Platform admin and observability

- **Superadmin console** (`/admin`): workspaces with plan/suspend, users and password resets, per-workspace errors and audit log, email diagnostics with resend, registration tokens, maintenance banners (shown to every user through `/api/system/banner/`), feedback triage, manual platform invoices, and demo leads.
- **Errors**: unhandled 500s from DRF views go to `ErrorLog`. Sentry is enabled when `SENTRY_DSN` is set. Non-DRF views (allauth, public pay/sign/session pages) only reach container logs and Sentry.
- **Health**: the 30-minute site and certificate check runs from inside the host. It was added after a certbot renewal failed silently in September 2026.
- **Django admin** at `/django-admin/` for raw data, periodic tasks and task results.

18 Operations

	Local	Production
Start	<code>docker compose up --build</code>	<code>git pull</code> → <code>make deploy</code> / <code>deploy-backend</code> / <code>deploy-frontend</code>
Compose file	<code>docker-compose.yml</code>	<code>docker-compose.prod.yml</code> (always)
Settings	<code>config.settings.local</code> (DEBUG)	<code>config.settings.production</code>
Frontend	Vite on <code>:5173</code> , proxying <code>/api</code> , <code>/accounts</code> , <code>/django-admin</code> , <code>/static</code>	Static build served by nginx
Email	Mailpit (nothing leaves the machine)	SES SMTP behind <code>DemoSafeEmailBackend</code>
Files	MinIO (S3-compatible)	AWS S3, private, presigned
TLS	none	Let's Encrypt via certbot webroot

Keys to back up: `SECRET_KEY` and `FIELD_ENCRYPTION_KEY`. Losing the second makes every stored Stripe key undecryptable, and it must never be shared between environments. Schema changes ship as Django migrations, which the backend entrypoint applies on start.

19 Known gaps and risks

Gap	Impact	Suggested next step
high RLS policies not enforced (§3)	Tenant isolation depends on every queryset filter being right	Non-owner DB role or <code>FORCE_RLS</code> ; extend to all workspace tables
high Google OAuth app in Testing status	Every coach's calendar connection silently dies weekly	Complete Google verification for the Calendar scope
med Zoom on shared S2S account	All workspaces' meetings under one Zoom identity	Resolve the OAuth redirect issue with Zoom; retire <code>ZOOM_OAUTH_ENABLED</code>
high Superadmin and Django admin see client data	Platform staff can read client names, emails, invoice amounts and notes, beyond troubleshooting needs	Redact superadmin responses to ids/status/counts; unregister client models from Django admin (PHASE2 DB-4/5)
med OnlyOffice file endpoints not bound to a workspace	With <code>ONLYOFFICE_JWT_SECRET</code> unset, a document UUID is enough to read or overwrite it	Require the secret in prod and match the token to the document (PHASE2 DB-6)
med RDS backups kept only 1 day	A problem noticed after a day can't be rolled back; <code>make_backup</code> dumps the unused container	Raise retention to 7–14 days, deletion protection, snapshot before deploys (PHASE2 P2-1)
med Single EC2 host	No app-tier failover; deploys restart services	Staging + rollback pipeline (PHASE2 P2-4)
med Tests use the live broker	Smoke test sends Celery tasks to the dev worker	<code>CELERY_TASK_ALWAYS_EAGER</code> in a test settings module
low <code>/portal</code> route is a dead stub	Confusing next to <code>/client-portal</code>	Redirect or delete
low <code>Assessment.visible_to_client</code> unused	Field implies a portal feature that doesn't exist	Build the endpoint or drop the field
low <code>late</code> status never set	Dead state in the lifecycle	Remove, or set it from the reminder job
low <code>dj-stripe</code> unused	Dead dependency and webhook route	Remove until platform billing is automated
low Errors outside DRF not in ErrorLog	Public page failures invisible in superadmin	Django-level exception middleware

The full multi-workspace readiness plan, with status per item, is in `PHASE2.md` . Docs out of date: `CLAUDE.md` §6–7 still say "5 scheduled jobs" and "no AI/LLM integration". There are 8 jobs, and Anthropic powers session-note drafts. `system_design.md` predates the OTP portal login, email logging, the reschedule workflow and the Zoom changes. This document reflects the code as of October 8, 2026.